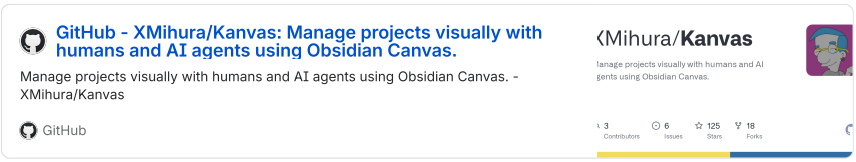
Kanvas como herramienta de gestión de agentes IA

Versión: 1.0
Fecha: 14 de marzo de 2026
Audiencia: Tech Leads, Developers

---

1. Qué es Kanvas y por qué Iberpixel lo usa

Kanvas es una herramienta de gestión visual de tareas ~~construida sobre **Obsidian**~~ que permite a los agentes de IA (principalmente Claude Code) gestionar su propio flujo de trabajo de forma autónoma.



Componentes del sistema

Componente	Función	Responsable
Obsidian	Aplicación base que visualiza el canvas	Humano (para visualización)
Canvas (.canvas file)	Fichero JSON que almacena estado de tareas	Sistema (Git versionado)
canvas-tool.py	Script CLI que el agente usa para gestionar tareas	Agente IA
File Watcher	Plugin de Obsidian que sincroniza cambios en tiempo real	Obsidian

Por qué Kanvas en lugar de Jira directo

Problema con Jira + Agentes:

- La API de Jira es compleja para que el agente la use de forma autónoma
- Requiere autenticación, permisos, configuración de webhooks
- Latencia en sincronización
- El agente no puede ver el estado visual del tablero

Solución con Kanvas:

- Fichero `.canvas` en el propio repositorio (versionado con Git)
- El agente lee/escribe con un script Python simple
- Obsidian visualiza el canvas en tiempo real para el humano
- Sincronización instantánea (file watcher)

- Sin dependencias externas ni APIs complejas

Flujo de información

```
Humano crea tareas → Obsidian → Project.canvas (Git)
                                        ↓
                            Agente lee con canvas-tool.py
                                        ↓
                            Agente actualiza estado (start/finish)
                                        ↓
                            File watcher detecta cambios
                                        ↓
                            Obsidian actualiza visualización

```

---

2. Instalación y configuración inicial

Prerequisitos

1. **Obsidian instalado**
   Descargar desde: https://obsidian.md
2. **Plugin: Obsidian Canvas**
   Viene incluido en Obsidian por defecto, verificar que está habilitado en Settings → Core plugins
3. **Python 3.8+**
   Verificar: `python3 --version`
4. **canvas-tool.py**
   Script corporativo de Iberpixel (disponible en repositorio interno)

Setup paso a paso

Paso 1: Crear vault de Obsidian para el proyecto

```
# Opción A: Usar el repositorio del proyecto como vault
# (recomendado para sincronización Git automática)
cd /ruta/proyecto
# Abrir esta carpeta como vault en Obsidian

# Opción B: Vault separado con enlace simbólico
mkdir ~/obsidian-vaults/proyecto-nombre
ln -s ~/obsidian-vaults/proyecto-nombre /ruta/proyecto/.obsidian

```

Recomendación: Opción A (vault = repositorio) para que el `.canvas` esté versionado con el resto del código.

Paso 2: Instalar canvas-tool.py en el proyecto

```
# Copiar script al proyecto
cp /ruta/corporativa/canvas-tool.py /ruta/proyecto/canvas-tool.py

# Dar permisos de ejecución
chmod +x canvas-tool.py

# Verificar que funciona
python3 canvas-tool.py --help

```

Salida esperada:

```
Usage: canvas-tool.py <canvas-file> <command> [options]

Commands:
  start <key>           Mover tarea a estado "Doing" (naranja)
  finish <key>          Mover tarea a estado "Done" (verde)
  block <key>           Mover tarea a estado "Blocked" (rojo)
  add-dep <key> <dep>   Añadir dependencia entre tareas
  list                  Listar todas las tareas y su estado
  status <key>          Ver estado actual de una tarea
```

```
10
```

Paso 3: Configurar file watcher en Obsidian

1. Abrir Obsidian → Settings → Community plugins
2. Browse → Buscar "File Watcher" o "Auto Refresh"
3. Instalar y habilitar
4. Configurar para que observe ficheros `.canvas`

Configuración recomendada:

- Watch file extensions: `.canvas`
- Auto-refresh interval: 1000ms (1 segundo)
- Notify on changes: No (silencioso)

Paso 4: Crear el canvas inicial del proyecto

1. En Obsidian: File → New canvas
2. Nombrar según nomenclatura del proyecto: `NombreProyecto.canvas`
   Ejemplo: `MinimalIso.canvas`, `SalesPulse.canvas`
3. Ubicación: raíz del vault (repositorio)

Paso 5: Verificar sincronización

```
# En terminal, ejecutar:
python3 canvas-tool.py NombreProyecto.canvas list

# Salida esperada (vacío si no hay tareas):
# No tasks found in canvas

```

Si funciona → setup completo ✅

---

3. Ciclo de estados y colores

Kanvas usa **colores** para representar estados de tareas, siguiendo el estándar corporativo de Iberpixel.

Mapa de estados

Color	Estado	Significado	Quién puede moverlo aquí
Morado	To Design	Tarea pendiente de especificación	Humano (diseñador/PM)
Rojo	Blocked	Bloqueada por dependencias no resueltas	Sistema automático o humano
Naranja	Doing	En implementación activa	Agente con `start`
Cyan	Review	Implementada, pendiente de	Agente con `finish`

		revisión humana	
Verde	Done	Completada y aprobada	Humano tras revisar

Flujo completo de una tarea

```
[Morado: To Design]
        ↓
   (Humano escribe spec)
        ↓
[Rojo: Blocked] ← Si tiene dependencias sin resolver
        ↓
   (Dependencias resueltas automáticamente)
        ↓
[Rojo: Blocked → listo para start]
        ↓
   (Agente ejecuta: start KEY)
        ↓
[Naranja: Doing]
        ↓
   (Agente implementa código)
        ↓
   (Agente ejecuta: finish KEY)
        ↓
[Cyan: Review]
        ↓
   (Humano revisa y aprueba)
        ↓
[Verde: Done]

```

Reglas de transición de estados

De Morado a Rojo:

- Manualmente por el humano cuando la spec está lista
- Automáticamente si se crea con dependencias definidas

De Rojo a Naranja:

- Solo el agente con `python3 canvas-tool.py start KEY`
- Solo si NO tiene dependencias bloqueantes
- Si tiene dependencias, debe esperar a que se resuelvan

De Naranja a Cyan:

- Solo el agente con `python3 canvas-tool.py finish KEY`
- Solo después de completar implementación y verificaciones

De Cyan a Verde:

- Solo el humano, manualmente en Obsidian
- Después de revisar código, verificar AC, aprobar PR

De cualquier estado a Rojo:

- Manualmente si se detecta un bloqueo
- Automáticamente por el sistema si aparece una nueva dependencia

---

4. Uso de canvas-tool.py por el agente

Comandos principales

`start <key>` - Iniciar trabajo en una tarea

Qué hace:

1. Verifica que la tarea existe
2. Verifica que NO está bloqueada por dependencias
3. Cambia el color a naranja (Doing)
4. Actualiza el fichero .canvas

Cuándo usarlo:
Al comenzar a trabajar en una tarea, antes de leer la spec.

Ejemplo:

```
python3 canvas-tool.py MinimalIso.canvas start MIM-001

```

Salida esperada:

```
✔ Task MIM-001 moved to Doing (orange)
✔ Canvas updated successfully

```

Errores posibles:

```
✗ Error: Task MIM-001 is blocked by dependencies: MIM-100
→ Solución: Esperar a que MIM-100 esté en Done (verde)

✗ Error: Task MIM-001 not found in canvas
→ Solución: Verificar que el KEY es correcto

```

`finish <key>` - Completar una tarea

Qué hace:

1. Verifica que la tarea está en estado Doing (naranja)
2. Cambia el color a cyan (Review)
3. Desbloquea automáticamente tareas dependientes
4. Actualiza el fichero .canvas

Cuándo usarlo:
Después de implementar el código, actualizar la spec con Execution Result, hacer commit.

Ejemplo:

```
python3 canvas-tool.py MinimalIso.canvas finish MIM-001

```

Salida esperada:

```
✔ Task MIM-001 moved to Review (cyan)
✔ Unblocked dependent tasks: MIM-002, MIM-003
✔ Canvas updated successfully

```

Errores posibles:

```
✗ Error: Task MIM-001 is not in Doing state
```

```
→ Solución: Ejecutar start primero

✗ Error: Cannot finish task with pending verifications
→ Solución: Ejecutar verificaciones y corregir errores

```

`add-dep <key> <dependency>` - **Añadir dependencia**

Qué hace:

1. Crea una flecha visual desde `<key>` hacia `<dependency>` en el canvas
2. Bloquea `<key>` si `<dependency>` no está en Done

Cuándo usarlo:
Cuando una tarea requiere que otra se complete primero.

Ejemplo:

```
python3 canvas-tool.py MinimalIso.canvas add-dep MIM-002 MIM-001

```

Interpretación:
"MIM-002 depende de MIM-001" → MIM-002 quedará bloqueada hasta que MIM-001 esté Done.

Salida esperada:

```
✔ Dependency added: MIM-002 → MIM-001
✔ Task MIM-002 blocked until MIM-001 is complete

```

⚠️ BUG CONOCIDO Y CORRECCIÓN:

El script original tenía las dependencias invertidas. Verificar que tu versión de `canvas-tool.py` tiene esta corrección:

```
# ✗ VERSIÓN INCORRECTA (bug)
def add_dependency(canvas_file, task_key, dependency_key):
    # Invertía la dirección de la flecha
    create_edge(dependency_key, task_key)  # MAL

# ✔ VERSIÓN CORREGIDA
def add_dependency(canvas_file, task_key, dependency_key):
    # Flecha desde task_key HACIA dependency_key
    create_edge(task_key, dependency_key)  # BIEN

```

Si tienes la versión con el bug, actualizar desde el repositorio corporativo.

`list` - **Listar todas las tareas**

Qué hace:
Muestra todas las tareas del canvas con su estado actual.

Ejemplo:

```
python3 canvas-tool.py MinimalIso.canvas list

```

Salida esperada:

```
Tasks in MinimalIso.canvas:

[DONE]     MIM-100  Epic: Base design system
[REVIEW]   MIM-001  User registration component
[DOING]    MIM-002  Login screen
[BLOCKED]  MIM-003  Dashboard layout (deps: MIM-002)
[TODO]     MIM-004  Profile settings
```

```
```

`status <key>` - Estado de una tarea específica

Qué hace:

Muestra información detallada de una tarea.

Ejemplo:

```
python3 canvas-tool.py MinimalIso.canvas status MIM-002

```

Salida esperada:

```
Task: MIM-002 - Login screen
Status: DOING (orange)
Dependencies: MIM-001 (DONE ✔)
Blocked by: None
Blocks: MIM-003, MIM-005
Spec: .ai/specs/MIM-002-login-screen.md

```

---

5. Gestión de dependencias

Cómo funciona el sistema de dependencias

Concepto:

Una tarea A "depende de" una tarea B significa que A NO puede comenzar hasta que B esté completada (verde).

Representación visual en Obsidian:

Flecha desde A hacia B.

Bloqueo automático:

Cuando se añade una dependencia, el sistema:

1. Verifica el estado de la tarea dependiente
2. Si NO está en Done (verde) → bloquea la tarea dependiente (rojo)
3. Cuando la tarea base pasa a Done → desbloquea automáticamente

Ejemplo completo de dependencias

Escenario:

- Epic MIM-100: Sistema de diseño base
- Subtarea MIM-001: Componente de botón
- Subtarea MIM-002: Formulario de login (requiere botón)

Configuración:

```
# MIM-002 depende de MIM-001
python3 canvas-tool.py MinimalIso.canvas add-dep MIM-002 MIM-001

# MIM-001 depende de MIM-100 (Epic)
python3 canvas-tool.py MinimalIso.canvas add-dep MIM-001 MIM-100

```

Resultado:

MIM-100 (rojo) → MIM-001 (rojo) → MIM-002 (rojo)

Flujo:

1. Se completa MIM-100 → pasa a verde
2. Sistema desbloquea automáticamente MIM-001 → puede pasar a naranja con `start`
3. Se completa MIM-001 → pasa a verde
4. Sistema desbloquea automáticamente MIM-002 → puede pasar a naranja con `start`

Bug de dependencias invertidas (RESUELTO)

Síntoma del bug:
Al ejecutar `add-dep MIM-002 MIM-001`, la flecha aparecía en dirección contraria (de MIM-001 hacia MIM-002).

Causa:
Error en la función `create_edge()` del script original que invertía `source` y `target`.

Corrección aplicada:

```
# Antes (bug):
{
  "id": edge_id,
  "fromNode": dependency_key,  # ← Invertido
  "toNode": task_key,
  ...
}

# Después (correcto):
{
  "id": edge_id,
  "fromNode": task_key,  # ← Correcto
  "toNode": dependency_key,
  ...
}

```

Cómo verificar que tienes la versión corregida:

```
# Crear dos nodos de prueba en Obsidian: TEST-A, TEST-B
# Ejecutar:
python3 canvas-tool.py Test.canvas add-dep TEST-A TEST-B

# Verificar en Obsidian que la flecha va desde TEST-A hacia TEST-B
# Si va al revés → actualizar canvas-tool.py

```

Desbloqueo automático de dependencias

Cuándo ocurre:
Cuando una tarea pasa a Done (verde), el sistema automáticamente:

1. Busca todas las tareas que dependen de ella
2. Para cada una, verifica si todas sus dependencias están resueltas
3. Si sí → cambia de rojo a rojo-listo (visualmente rojo, pero puede recibir `start`)

Ejemplo:

```
# Estado inicial:
MIM-001 (verde) ← MIM-002 (rojo, dependiente)

# Humano aprueba MIM-001 y lo pasa a verde manualmente en Obsidian

# Sistema detecta el cambio vía file watcher

# Al ejecutar:
python3 canvas-tool.py MinimalIso.canvas start MIM-002
# ✔ Ahora funciona, porque MIM-001 está Done

```

---

6. Integración con file watcher de Obsidian

Qué es el file watcher

Plugin de Obsidian que detecta cambios en ficheros del vault y refresca la visualización automáticamente.

Sin file watcher:
Después de que el agente ejecuta `start` o `finish`, Obsidian no actualiza el canvas hasta que cierres y reabras el fichero manualmente.

Con file watcher:
Los cambios se reflejan en 1-2 segundos automáticamente.

Configuración recomendada

Plugin: "Auto Refresh" o "File Watcher"
Settings:

```
Watch patterns: "*.canvas"
Refresh interval: 1000ms
Auto-scroll to changes: false
Notify on refresh: false

```

Verificación de sincronización

Test manual:

1. Abrir `Proyecto.canvas` en Obsidian
2. En terminal, ejecutar: `python3 canvas-tool.py Proyecto.canvas start MIM-001`
3. Observar Obsidian → el nodo MIM-001 debe cambiar a naranja en 1-2 segundos

Si NO cambia:

- Verificar que file watcher está habilitado
- Reducir refresh interval a 500ms
- Reiniciar Obsidian

Conflictos de edición simultánea

Problema:
Si el humano edita el canvas en Obsidian AL MISMO TIEMPO que el agente ejecuta `start` / `finish`, puede haber conflicto de escritura.

Solución:

1. El humano NO debe editar manualmente estados de tareas (colores)
2. Solo editar: posiciones de nodos, textos, añadir/eliminar nodos
3. Dejar la gestión de estados al agente vía `canvas-tool.py`

Si ocurre conflicto:
Git detectará el conflicto en el fichero `.canvas` (es JSON). Resolver manualmente eligiendo la versión correcta.

---

7. Nomenclatura de tareas (integración con Jira)

Formato estándar de KEYs

Iberpixel usa nomenclatura de Jira para mantener trazabilidad:

Formato: `PROYECTO-NNN`

Ejemplos:

- `MIM-001`, `MIM-002`, `MIM-003` (MinimalIso)
- `SP-045`, `SP-046` (SalesPulse)
- `WEB-012` (Web corporativa)

Jerarquía: Epics y Subtareas

Epics (KEY-100, KEY-200, ...):
Tareas de diseño, configuración base, decisiones arquitectónicas.

Ejemplos:

- `MIM-100`: Epic de sistema de diseño base
- `MIM-200`: Epic de configuración de autenticación
- `MIM-300`: Epic de estructura de datos

Subtareas (KEY-001, KEY-002, ...):
Implementaciones específicas que dependen de un Epic.

Ejemplos:

- `MIM-001`: Componente de botón (depende de MIM-100)
- `MIM-002`: Formulario de login (depende de MIM-001)

Creación de tareas en Obsidian

Paso a paso:

1. Click derecho en canvas → Add note
2. Nombrar el nodo: `KEY - Descripción corta`
   Ejemplo: `MIM-001 - Componente de botón`
3. Asignar color inicial según estado:
   - Morado si está pendiente de spec
   - Rojo si tiene spec pero dependencias bloqueantes
4. Añadir dependencias con `add-dep` si es necesario

Ejemplo completo de creación:

```
# 1. Crear nodos en Obsidian visualmente:
#    - MIM-100 - Sistema de diseño base (morado)
#    - MIM-001 - Componente de botón (morado)
#    - MIM-002 - Formulario de login (morado)

# 2. Añadir dependencias vía CLI:
python3 canvas-tool.py MinimalIso.canvas add-dep MIM-001 MIM-100
python3 canvas-tool.py MinimalIso.canvas add-dep MIM-002 MIM-001

# 3. Cuando MIM-100 tenga spec lista, cambiar manualmente a rojo en Obsidian

# 4. Agente puede entonces ejecutar:
```

```
python3 canvas-tool.py MinimalIso.canvas start MIM-100

```

Sincronización con Jira (opcional)

Kanvas NO sincroniza automáticamente con Jira. Es un sistema independiente.

Flujo recomendado:

1. Crear tickets en Jira con KEYs normales
2. Replicar manualmente esos KEYs en Kanvas
3. Mantener Jira para gestión de proyecto (roadmap, sprints)
4. Usar Kanvas para gestión operativa del agente (día a día)

Sincronización manual:
Al completar una tarea (verde en Kanvas), marcar el ticket en Jira como Done también.

---

8. Workflow completo: Ejemplo real paso a paso

Escenario

Implementar **MIM-001: Componente de botón** que depende de **MIM-100: Sistema de diseño base**.

Estado inicial

```
MIM-100 (verde - completado previamente)
   ↑
   |
MIM-001 (rojo - tiene spec, bloqueado por dependencia)

```

Paso 1: Humano verifica que la dependencia está resuelta

```
python3 canvas-tool.py MinimalIso.canvas status MIM-001

```

Salida:

```
Task: MIM-001 - Componente de botón
Status: BLOCKED (red)
Dependencies: MIM-100 (DONE ✔)
Blocked by: None (ready to start)
Spec: .ai/specs/MIM-001-componente-boton.md

```

✔ MIM-100 está Done → MIM-001 puede comenzar.

Paso 2: Agente ejecuta start

```
python3 canvas-tool.py MinimalIso.canvas start MIM-001

```

Salida:

```
✔ Task MIM-001 moved to Doing (orange)
✔ Canvas updated successfully

```

Obsidian actualiza automáticamente: MIM-001 cambia a naranja.

Paso 3: Agente lee la spec

```
# Agente ejecuta internamente:
cat .ai/specs/MIM-001-componente-boton.md

```

Paso 4: Agente implementa el código

El agente:

1. Crea el componente según la spec
2. Implementa todos los Acceptance Criteria
3. Ejecuta verificaciones (type-check, lint, tests, build)

Paso 5: Agente actualiza la spec con Execution Result

Añade al final de `.ai/specs/MIM-001-componente-boton.md`:

```
## Execution Result

**Status:** PASS
**Date:** 2026-03-14
**Agent:** Claude Code
**Implementation notes:**
- Componente creado en `src/components/button.tsx`
- Variantes: primary, secondary, outline
- Responsive: funciona en mobile y desktop
- Integrado con sistema de tokens (D-04)

**Verification results:**
- Type check: PASS ✔
- Linting: PASS ✔
- Build: PASS ✔
- Tests: N/A (sin tests unitarios aún)

```

Paso 6: Agente hace commit

```
git add src/components/button.tsx
git add .ai/specs/MIM-001-componente-boton.md
git commit -m "feat(MIM-001): implementar componente de botón con variantes

- Componente Button con props tipadas
- Variantes: primary, secondary, outline
- Integración con tokens de diseño
- Verificaciones pasadas"

```

Paso 7: Agente ejecuta finish

```
python3 canvas-tool.py MinimalIso.canvas finish MIM-001

```

Salida:

```
✔ Task MIM-001 moved to Review (cyan)
✔ Unblocked dependent tasks: MIM-002
✔ Canvas updated successfully

```

Obsidian actualiza: MIM-001 cambia a cyan.

Paso 8: Humano revisa

Developer humano:

1. Abre PR en Git
2. Revisa el código generado
3. Verifica que cumple los AC de la spec
4. Prueba manualmente el componente
5. Aprueba la PR

Paso 9: Humano marca como Done

En Obsidian:

1. Click derecho en nodo MIM-001
2. Change color → Verde

Resultado:
MIM-001 pasa a Done (verde).

Efecto en dependencias:
MIM-002 (que dependía de MIM-001) ahora puede ejecutar `start`.

---

9. Troubleshooting común

Error: "Task not found in canvas"

Causa:
El KEY no existe en el fichero .canvas o está mal escrito.

Solución:

```
# Listar todas las tareas para ver KEYs exactos
python3 canvas-tool.py Proyecto.canvas list

# Verificar que el KEY coincide exactamente (case-sensitive)

```

Error: "Task is blocked by dependencies"

Causa:
Intentas ejecutar `start` en una tarea que tiene dependencias no resueltas.

Solución:

```
# Ver qué dependencias están bloqueando
python3 canvas-tool.py Proyecto.canvas status KEY

# Ejemplo de salida:
# Dependencies: MIM-100 (TODO ✗), MIM-101 (DONE ✔)
# Blocked by: MIM-100

# Completar primero MIM-100

```

El canvas no se actualiza en Obsidian después de ejecutar comandos

Causa:
File watcher no está activo o tiene intervalo de refresco muy largo.

Solución:

1. Verificar que file watcher está habilitado en Obsidian
2. Reducir intervalo a 500ms o 1000ms
3. Reiniciar Obsidian
4. Si persiste: cerrar y reabrir el canvas manualmente

Conflicto de merge en .canvas después de pull

Causa:

Dos personas o dos agentes modificaron el canvas simultáneamente.

Solución:

```
# El fichero .canvas es JSON, Git lo marca como conflicto

# Opción A: Aceptar versión remota
git checkout --theirs Proyecto.canvas
git add Proyecto.canvas

# Opción B: Aceptar versión local
git checkout --ours Proyecto.canvas
git add Proyecto.canvas

# Opción C: Resolver manualmente editando el JSON
# (complicado, no recomendado)

```

Dependencias invertidas (flechas en dirección contraria)

Causa:

Versión antigua de `canvas-tool.py` con el bug de dependencias invertidas.

Solución:

```
# Actualizar canvas-tool.py desde el repositorio corporativo
cp /ruta/corporativa/canvas-tool.py ./canvas-tool.py

# Verificar versión corregida:
grep "fromNode.*task_key" canvas-tool.py
# Debe aparecer: fromNode: task_key (no dependency_key)

```

Agent no puede escribir en .canvas (permisos)

Causa:

Fichero .canvas tiene permisos restrictivos.

Solución:

```
# Verificar permisos
ls -l Proyecto.canvas

# Dar permisos de escritura
chmod 644 Proyecto.canvas

```

---

10. Mejores prácticas

Para el humano (Tech Lead / Developer)

1. **Crear tareas en Obsidian antes de que el agente las necesite**
   El agente no crea tareas, solo las gestiona.
2. **Usar colores consistentemente**
   Morado = sin spec, Rojo = bloqueado, Naranja/Cyan/Verde = flujo del agente.
3. **No editar colores manualmente mientras el agente trabaja**
   Dejar que `canvas-tool.py` gestione estados automáticamente.
4. **Revisar tareas en cyan rápidamente**
   Cuando el agente mueve a cyan, revisar pronto para no bloquear el flujo.

5. **Mantener Kanvas y Jira sincronizados manualmente**
   Si usas Jira, actualizar manualmente cuando una tarea cambia de estado.

Para el agente (en AGENTS.md)

Incluir estas reglas en el fichero `AGENTS.md` del proyecto:

```
## Reglas de Kanvas para el Agente

1. Antes de comenzar cualquier tarea, ejecutar:
   `python3 canvas-tool.py Proyecto.canvas start KEY`

2. Solo trabajar tareas que puedan hacer `start` exitosamente.
   Si devuelve error de dependencias bloqueantes, esperar.

3. Después de implementar y hacer commit, ejecutar:
   `python3 canvas-tool.py Proyecto.canvas finish KEY`

4. No modificar el fichero .canvas manualmente.
   Usar siempre canvas-tool.py para cambios de estado.

5. Si una tarea no tiene spec en .ai/specs/KEY-*.md,
   preguntar al humano antes de continuar.

```

Estructura recomendada del canvas

Organización visual en Obsidian:

```
[Epics en la parte superior]
    MIM-100 (verde)    MIM-200 (rojo)    MIM-300 (morado)
         ↓                 ↓                 ↓
[Subtareas dependientes debajo]
    MIM-001 (cyan)     MIM-201 (rojo)    (vacío aún)
    MIM-002 (naranja)
    MIM-003 (rojo)

```

Ventajas:

- Visualmente claro qué subtareas dependen de qué epic
- Fácil identificar bloqueadores (rojo)
- Fácil ver qué está en progreso (naranja)

---

11. Extensiones y automatizaciones futuras

Integración con Jira API (roadmap)

Objetivo:
Sincronización bidireccional automática entre Kanvas y Jira.

Implementación propuesta:

- Script Python que lee .canvas y actualiza tickets en Jira via API
- Webhook de Jira que actualiza .canvas cuando se cambia un ticket
- Intervalo de sincronización: cada 5 minutos

Estado: En análisis, no implementado aún.

Métricas automáticas del agente

Objetivo:
Generar métricas de productividad del agente desde el canvas.

Métricas propuestas:

- Tiempo promedio en estado Doing (naranja → cyan)
- Tasa de PASS vs FAIL en Execution Results
- Número de tareas bloqueadas por dependencias

Implementación:
Script Python que parsea .canvas y genera reporte.

Estado: Propuesta, pendiente de implementar.

Notificaciones automáticas

Objetivo:
Notificar al humano cuando el agente mueve una tarea a cyan (Review).

Canales posibles:

- Slack
- Discord
- Email

Estado: No implementado, prioridad baja.

---

12. Checklist de setup de Kanvas en proyecto nuevo

Para Tech Leads al iniciar un proyecto:

Instalación base

- [ ] Obsidian instalado
- [ ] Plugin Canvas habilitado
- [ ] Plugin File Watcher instalado y configurado
- [ ] Python 3.8+ disponible
- [ ] canvas-tool.py copiado al proyecto

Configuración del canvas

- [ ] Crear fichero `NombreProyecto.canvas` en raíz del repositorio
- [ ] Abrir canvas en Obsidian
- [ ] Verificar que file watcher detecta el fichero

Verificación funcional

- [ ] Ejecutar `python3 canvas-tool.py NombreProyecto.canvas list` → funciona
- [ ] Crear tarea de prueba en Obsidian
- [ ] Ejecutar `start` en tarea de prueba → color cambia a naranja
- [ ] Ejecutar `finish` en tarea de prueba → color cambia a cyan

Integración con el agente

- [ ] Añadir reglas de Kanvas a AGENTS.md
- [ ] Crear primera tarea real con spec

- [ ] Probar flujo completo: start → implementación → finish
- [ ] Verificar que Execution Result se actualiza correctamente

Control de versiones

- [ ] Fichero .canvas incluido en Git
- [ ] .gitignore NO excluye .canvas
- [ ] .obsidian/ excluido en .gitignore (configuración local)
- [ ] Commit inicial: `chore: setup Kanvas para gestión de tareas`

Referencias

- Obsidian oficial
- Obsidian Canvas documentation
- canvas-tool.py en repositorio corporativo
- AGENTS.md template corporativo

Última actualización: 14 de marzo de 2026
Próxima revisión: Al detectar nuevos bugs o añadir features a canvas-tool.py